

Kadane's Algorithm

Basics → Master: Theory, Proof, Variants & LeetCode Problem Bank

Kadane's is a single, small algorithm — but it's actually your first real 1D Dynamic Programming pattern in disguise. Understanding WHY it works (not just the four-line code) is what lets you recognize its many disguised forms: circular arrays, products instead of sums, 2D matrices, and counting problems. This guide goes deep on the proof, then widens out to every major variant.

Section	Covers
1. What & Why	The problem Kadane's solves, brute force -> $O(n)$
2. Core Theory & Proof	The local/global max recurrence, why greedy reset works
3. Java Templates	Standard, with indices, and all-negative handling
4. Problem Bank (Tiers 1-6)	22 curated problems, basics -> master
5. Fully Worked Examples	4 problems solved with full reasoning + code
6. The Variant Family	Circular, product, 2D matrix, k-concatenation
7. Common Mistakes	All-negative arrays, empty subarray edge cases, etc.
8. Practice Roadmap	Order to solve + spaced repetition schedule

1. What Kadane's Algorithm Solves & Why It Works

Kadane's Algorithm finds the maximum sum of a **contiguous** subarray in $O(n)$ time. The naive brute force checks every possible (start, end) pair and sums each — $O(n^2)$ (or $O(n^3)$ if you recompute each sum from scratch). Kadane's collapses this to a single pass using a simple but easy-to-doubt insight about when to abandon your current subarray.

The core question Kadane's answers at every index

At each position i , you ask exactly one question: **"Is it better to extend the previous subarray by including $\text{nums}[i]$, or is the previous subarray actively hurting me — should I start a brand new subarray at i ?"** That single decision, repeated once per element, is the entire algorithm.

How to recognize a Kadane's problem

- Keywords: "maximum subarray sum", "maximum contiguous sum", "largest sum", "maximum product subarray".
- You need a contiguous run (not subsequence) that optimizes some running accumulation (sum, product).
- The array can contain negative numbers — this is precisely what makes the problem non-trivial (if everything were positive, the answer is just the whole array's sum).

Kadane's is technically a 1D Dynamic Programming algorithm dressed up as a simple loop — $\text{dp}[i] = \max(\text{nums}[i], \text{dp}[i-1] + \text{nums}[i])$. Recognizing this DP shape is what lets you generalize to the product and 2D variants later.

2. Core Theory & Proof (don't skip this)

2.1 The recurrence

Define `currentMax` = the maximum sum of a subarray ENDING exactly at the current index (not just any subarray up to here — it MUST include the current element). Then:

```
currentMax[i] = max(nums[i], currentMax[i-1] + nums[i])
globalMax     = max(globalMax, currentMax[i]) // best seen across all i
```

- This is a genuine DP recurrence — `currentMax[i]` only depends on `currentMax[i-1]`, so you can collapse the DP array into a single running variable (that's why the final code looks like "just a loop").
- `currentMax[i-1] + nums[i]` is 'extend the previous best run'. `nums[i]` alone is 'abandon everything before, restart here'.
- Whichever is bigger IS the best subarray ending at `i` — there's no third option, because any subarray ending at `i` either includes `i-1` or doesn't (and if it doesn't, starting fresh at `i` is optimal since `nums[i]` alone is the best you can do without extending).

2.2 Why 'reset when negative' is a correct greedy, not a hack

The intuitive shortcut people memorize is: "if `currentMax` drops below 0, reset it to 0 before adding the next element." This IS mathematically the same recurrence above — here's why it's provably correct and not just a heuristic:

- If `currentMax` (the best sum ending at the previous index) is negative, then for ANY future element `x`, $(\text{currentMax} + x) < x$ — carrying a negative prefix can only ever make the next sum worse, never better.
- So the moment `currentMax` goes negative, it is **strictly dominated** by starting fresh — keeping it around is provably never beneficial, which is what makes the greedy reset safe rather than just 'usually good'.
- This is the same logic as Container With Most Water's 'move the shorter pointer' — a local greedy choice that's provably never wrong, not a lucky heuristic.

2.3 Why it's $O(n)$ and $O(1)$ space

- Single pass, one comparison + one addition per element — $O(n)$ time.
- Only two running variables (`currentMax`, `globalMax`) needed — $O(1)$ space, no DP array required once you see the recurrence only needs the previous value.

3. Java Templates

3.1 Standard Kadane's (sum, all-negative safe)

```
public int maxSubArray(int[] nums) {
    int currentMax = nums[0];
    int globalMax = nums[0];

    for (int i = 1; i < nums.length; i++) {
        // extend previous run, or start fresh at i -- whichever is bigger
        currentMax = Math.max(nums[i], currentMax + nums[i]);
        globalMax = Math.max(globalMax, currentMax);
    }
    return globalMax;
}
```

Initializing both to `nums[0]` (NOT 0) is what makes this safe for all-negative arrays — starting at 0 would incorrectly allow an 'empty subarray' with sum 0 to beat a genuinely negative answer.

3.2 Kadane's with the actual subarray indices (not just the sum)

```
public int[] maxSubArrayWithIndices(int[] nums) {
    int currentMax = nums[0], globalMax = nums[0];
    int start = 0, bestStart = 0, bestEnd = 0;

    for (int i = 1; i < nums.length; i++) {
        if (nums[i] > currentMax + nums[i]) {
            currentMax = nums[i];
            start = i; // restart candidate window here
        } else {
            currentMax += nums[i]; // extend
        }
        if (currentMax > globalMax) {
            globalMax = currentMax;
            bestStart = start;
            bestEnd = i;
        }
    }
    return new int[]{globalMax, bestStart, bestEnd};
}
```

4. Problem Bank — Tier 1 to Tier 6 (22 problems)

Kadane's has fewer 'basic' problems than sliding window/two pointers because it IS one specific algorithm — the depth here is mostly in recognizing its variants (Tiers 3-6), not in Tier 1-2 repetition.

Tier 1 — Core Kadane's

LC#	Problem	Diff.	Key Concept / Theory
53	Maximum Subarray	Med	THE canonical problem — implement the standard template from Section 3.1 cold.
1913	Maximum Product Difference Between Two Pairs	Easy	Not Kadane's itself, but a good warmup in max/min running-value tracking.
custom	Maximum Subarray with Indices	-	Re-solve 53 but return the actual start/end indices — forces you to internalize Section 3.2.

Tier 2 — Minimum Subarray Sum (the mirror problem)

LC#	Problem	Diff.	Key Concept / Theory
custom	Minimum Subarray Sum	Med	Flip every comparison in the template (min instead of max) — proves you understand the recurrence, not just memorized code.
1191	K-Concatenation Maximum Sum	Med	Kadane's on array concatenated 1 or 2 times, plus total-sum * (k-2) if total sum is positive — combines Kadane's with a math shortcut for large k.

Tier 3 — Counting / Existence Variants

LC#	Problem	Diff.	Key Concept / Theory
152	Maximum Product Subarray	Med	Track BOTH currentMax AND currentMin at each step, since a negative number can flip a very negative running product into the new maximum.
918	Maximum Sum Circular Subarray	Med	Answer is max(standard Kadane's, totalSum - minSubarraySum via Kadane's) — the circular wrap case is 'total minus the worst middle chunk'.
1191	(Revisit) K-Concatenation Maximum Sum	Med	Revisit after 918 — both combine Kadane's with a whole-array sum trick.

Tier 4 — 2D / Matrix Kadane's

LC#	Problem	Diff.	Key Concept / Theory
363	Max Sum of Rectangle No Larger Than K	Hard	Fix a pair of column boundaries, collapse rows between them into a 1D array via prefix sums, then run Kadane's-adjacent logic (with a TreeSet for the 'no larger than K' constraint).

LC#	Problem	Diff.	Key Concept / Theory
custom	Maximum Sum Rectangle in a 2D Matrix	Hard	Classic interview extension: fix top/bottom row boundaries ($O(n^2)$ pairs), collapse columns into a 1D array by summing between those rows, run standard Kadane's on that 1D array ($O(m)$) — $O(n^2 \cdot m)$ total.

Tier 5 — Kadane's Disguised Inside a Larger Problem

LC#	Problem	Diff.	Key Concept / Theory
121	Best Time to Buy and Sell Stock	Easy	Reframe as: track minPriceSoFar, and at each day compute price - minPriceSoFar — this IS Kadane's on the array of day-to-day differences.
134	Gas Station	Med	If total gas $\geq$ total cost, a valid start exists; find it via a Kadane's-style reset whenever the running tank goes negative.
845	Longest Mountain in Array	Med	Not sum-based, but same 'track a running state, reset when the pattern breaks' shape as Kadane's — good pattern-transfer check.

Tier 6 — Master Tier

LC#	Problem	Diff.	Key Concept / Theory
978	Longest Turbulent Subarray	Med	Track two running lengths (ending in an 'up' vs 'down' comparison) simultaneously — a two-state extension of the single running value in standard Kadane's.
1746	Maximum Subarray Sum After One Operation	Med	Extend the DP state to TWO dimensions per index: best sum with 0 elements squared, and best sum with exactly 1 element squared — direct generalization of the currentMax recurrence.
198	House Robber (Non-Adjacent Max Sum)	Med	Different recurrence ($dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$) but same 'single running comparison replaces an $O(n)$ inner search' DP philosophy as Kadane's.

5. Fully Worked Examples (with reasoning)

Example A — 53. Maximum Subarray (the baseline, done twice: DP array then $O(1)$)

Problem: Find the contiguous subarray with the largest sum.

```
// Step 1: see it as an explicit DP array first (for understanding)
public int maxSubArrayDP(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    dp[0] = nums[0];
    int globalMax = dp[0];

    for (int i = 1; i < n; i++) {
        dp[i] = Math.max(nums[i], dp[i - 1] + nums[i]);
        globalMax = Math.max(globalMax, dp[i]);
    }
    return globalMax;
}
```

Write the DP-array version first when learning, then collapse it. Skipping straight to the $O(1)$ version is exactly why people memorize Kadane's without understanding it.

Example B — 152. Maximum Product Subarray (why you need TWO running values)

Problem: Find the contiguous subarray with the largest product.

Reasoning

- With sums, a very negative running value is always bad to keep. With PRODUCTS, a very negative running value can become the best possible answer if multiplied by another negative number.
- So track both currentMax AND currentMin ending at each index. When you hit a negative number, the current max and min SWAP roles (multiplying by a negative flips the sign, so what was the minimum can become the new maximum).

```
public int maxProduct(int[] nums) {
    int currentMax = nums[0], currentMin = nums[0], globalMax = nums[0];

    for (int i = 1; i < nums.length; i++) {
        int num = nums[i];
        if (num < 0) {
            int temp = currentMax;
            currentMax = currentMin;
            currentMin = temp;          // swap before multiplying, since sign flips
        }
        currentMax = Math.max(num, currentMax * num);
        currentMin = Math.min(num, currentMin * num);
        globalMax = Math.max(globalMax, currentMax);
    }
    return globalMax;
}
```

Example C — 918. Maximum Sum Circular Subarray (Kadane's used twice)

Problem: Array is circular (end connects to start) — find the max sum of a circular subarray.

Reasoning

- There are exactly two cases: (1) the best subarray does NOT wrap around — that's just standard Kadane's. (2) it DOES wrap around, meaning it's everything EXCEPT some middle chunk — that remaining middle chunk must be the MINIMUM subarray sum, so the wrap-around answer is $\text{totalSum} - \text{minSubarraySum}$.
- Edge case: if the entire array is negative, the 'exclude the minimum subarray' trick would incorrectly exclude everything (leaving an empty array) — in that case just return the standard (non-circular) Kadane's answer, since the circular case can't help.

```
public int maxSubarraySumCircular(int[] nums) {
    int totalSum = 0, currentMax = 0, globalMax = nums[0], currentMin = 0, globalMin = nums[0];

    for (int num : nums) {
        totalSum += num;
        currentMax = Math.max(num, currentMax + num);
        globalMax = Math.max(globalMax, currentMax);
        currentMin = Math.min(num, currentMin + num);
        globalMin = Math.min(globalMin, currentMin);
    }

    if (globalMax < 0) return globalMax; // all-negative edge case
    return Math.max(globalMax, totalSum - globalMin);
}
```

Example D — 121. Best Time to Buy and Sell Stock (Kadane's in disguise)

Problem: Find the max profit from buying then selling one share, buy must happen before sell.

Reasoning

- Reframe: profit if you sell on day i = $\text{price}[i] - (\text{minimum price seen on any day before } i)$.
- Track `minPriceSoFar` as you scan once; at each day compute the profit if you sold today, keep the best.
- This is structurally identical to Kadane's 'best value using the current element, compared against the best baseline seen so far' — just phrased in business terms instead of sums.

```
public int maxProfit(int[] prices) {
    int minPriceSoFar = prices[0], maxProfit = 0;

    for (int i = 1; i < prices.length; i++) {
        maxProfit = Math.max(maxProfit, prices[i] - minPriceSoFar);
        minPriceSoFar = Math.min(minPriceSoFar, prices[i]);
    }
    return maxProfit;
}
```


6. The Variant Family — Map of Every Disguise

Variant	What Changes	Example
Standard sum	Base case	LC 53
Product	Track max AND min (sign flips)	LC 152
Circular array	$\max(\text{standard}, \text{total} - \text{min subarray})$	LC 918
2D matrix	Fix row/col boundaries, collapse to 1D, run Kadane's	LC 363 / Max Sum Rectangle
K-concatenation	Kadane's on 1-2 copies + math shortcut for large k	LC 1191
Reframed business problem	Same recurrence, different variable names	LC 121 (stock)
Constrained (one flip/operation)	Extra DP dimension for 'used the operation or not'	LC 1746

If you can map a brand-new problem to one row of this table before coding, you've actually learned Kadane's as a transferable pattern rather than a memorized snippet.

7. Common Mistakes

- **Initializing currentMax/globalMax to 0 instead of nums[0]:** silently breaks all-negative arrays by allowing a phantom 'empty subarray with sum 0' to win.
- **Forgetting the sign-flip swap in Maximum Product Subarray:** without swapping currentMax/currentMin before multiplying by a negative number, you lose the case where two negatives should combine into a large positive.
- **Applying standard Kadane's directly to a circular array:** you must explicitly handle the wrap-around case ($\text{totalSum} - \text{minSubarray}$) — the standard template alone misses subarrays that cross the array boundary.
- **Confusing 'subarray' with 'subsequence':** Kadane's only works for CONTIGUOUS runs. If a problem allows skipping elements (non-contiguous), that's DP with a different recurrence (like House Robber), not Kadane's.
- **Forgetting the all-negative edge case in circular variants:** if every element is negative, $\text{totalSum} - \text{minSubarray}$ would incorrectly produce an empty result — must fall back to standard Kadane's result in that case.
- **Not recognizing disguised Kadane's:** problems like Best Time to Buy and Sell Stock don't mention 'subarray' at all — missing the reframe ($\text{price}[i] - \text{minSoFar}$) means solving it the hard way with nested loops.

8. Practice Roadmap & Spaced Repetition

Suggested order (2-3 problems/day)

- **Days 1-2:** Tier 1 — implement 53 both ways (explicit DP array, then collapsed $O(1)$) before moving on.
- **Day 3:** Tier 2 — flip the recurrence for minimum subarray sum from scratch, no peeking at Tier 1 code.
- **Days 4-6:** Tier 3 — product and circular variants; these are the two most-tested twists in interviews.
- **Days 7-9:** Tier 4 — 2D matrix Kadane's; this is a genuinely different skill (row/column collapsing) worth its own dedicated time.
- **Days 10-12:** Tier 5 — disguised Kadane's; the goal here is recognition speed, not new coding technique.
- **Days 13-15:** Tier 6 — attempt cold, these extend the DP state itself.

Spaced repetition after first solve

- Day 1: solve with full struggle (aim 20-25 min before hints — Kadane's problems are usually shorter than sliding window/two pointer ones).
- Day 3: re-solve cold — can you state the recurrence out loud before writing code?
- Day 7: re-solve cold again; for variants, write which row of the Section 6 table this problem maps to.
- Day 21: final cold re-solve of 53, 152, and 918 back to back — if all three are fast, Kadane's is fully internalized.

Same method as your Sliding Window, Two Pointers, and ArrayList guides — struggle first, use this as the reference after.